Vienna Traffic Router
Heuristics System Documentation

src/heuristics/

April 2026

Contents

1 Overview **3**
- 1.1 Core Principle . . . 3
- 1.2 Design Goals . . . 3

2 Architecture: Two-Tier Design **4**
- 2.1 Tier 1: Static Multipliers . . . 4
- 2.2 Tier 2: Goal-Dependent Multipliers . . . 4
- 2.3 Tier 3: Node and Edge-Dependent Multipliers . . . 5

3 The Orchestrator: `combined.py` **6**
- 3.1 Module Structure . . . 6
- 3.2 Enabling/Disabling Heuristics . . . 6
- 3.3 Closure Construction Pseudocode . . . 6
- 3.4 Admissibility and Capping . . . 7

4 Heuristic Categories and Patterns **8**
- 4.1 Pattern 1: Lookup Tables (Static) . . . 8
  - 4.1.1 Example: `weather.py` . . . 8
  - 4.1.2 Example: `vehicle_type.py` . . . 8
- 4.2 Pattern 2: Conditional Penalties (Static + Dynamic) . . . 8
  - 4.2.1 Example: `safety.py` . . . 8
- 4.3 Pattern 3: Multi-Dimensional Lookup (Edge + Vehicle) . . . 9
  - 4.3.1 Example: `surface.py` . . . 9
- 4.4 Pattern 4: Geospatial Lookup (District-Based) . . . 9
  - 4.4.1 Example: `parking.py` . . . 9
- 4.5 Pattern 5: Time-Based Conditional (Hour, Day, Date) . . . 10
  - 4.5.1 Example: `time_of_day.py` . . . 10
  - 4.5.2 Example: `school_zones.py` . . . 10
- 4.6 Pattern 6: Proximity-Based (Geospatial Queries) . . . 10
  - 4.6.1 Example: `road_works.py` . . . 10
  - 4.6.2 Example: `markets.py` . . . 10
- 4.7 Pattern 7: Inference (Derived Conditions) . . . 10
  - 4.7.1 Example: `weather.py` — Black Ice Inference . . . 11

5 The 30 Heuristics: Full Reference **12**
5.1 Environmental (5 heuristics) . . . 12
5.2 Temporal (5 heuristics) . . . 12
5.3 Vehicle-Type (3 heuristics) . . . 12
5.4 Road Geometry and Condition (6 heuristics) . . . 12
5.5 Safety and Regulation (6 heuristics) . . . 13
5.6 Urban Features and Attractions (5 heuristics) . . . 13
5.7 Destination-Specific (3 heuristics) . . . 13

6 Adding a New Heuristic **14**

7 Performance Considerations **15**
7.1 Closure-Freeing Optimization . . . 15
7.2 Edge Proxy Strategy . . . 15
7.3 Capping Mechanism . . . 15

8 Integration with Pathfinding Algorithms **16**
8.1 Admissibility Requirements . . . 16

9 Configuration and Testing **16**
9.1 Environment Variables . . . 16
9.2 Testing a Heuristic . . . 16

10 Summary **18**
10.1 Key Design Points . . . 18
10.2 File Organization . . . 18
10.3 Code Style . . . 18

1 Overview

The heuristics system in `src/heuristics/` implements a sophisticated multi-dimensional cost model for the Vienna road network. It powers all eight pathfinding algorithms (BFS, DFS, UCS, Dijkstra, Greedy Best-First, A*, Weighted A*, Bidirectional A*) by providing informed estimates of distance-to-goal that account for 30+ real-world factors: weather, time of day, vehicle type, surface quality, safety, emissions, events, parking difficulty, and Vienna-specific local effects.

1.1 Core Principle

Each heuristic module returns a *multiplier* $m \in \mathbb{R}^+$ that scales the raw Haversine distance:

$$h(n, \text{goal}) = \text{haversine}(n, \text{goal}) \times \prod_{i=1}^{n} m_i$$

The multiplier is multiplicative, not additive, so a road penalized by weather (factor 1.5), surface (factor 1.2), and elevation (factor 1.1) incurs a combined penalty of $1.5 \times 1.2 \times 1.1 = 1.98$.

1.2 Design Goals

1. **Interactivity:** Heuristics must evaluate thousands of times per route (once per frontier node in A*) with sub-millisecond latency.
2. **Admissibility:** Static heuristics never overestimate true cost; weighted A* remains bounded-suboptimal via a hard cap.
3. **Modularity:** Each heuristic is independent, testable, and can be toggled on/off via the UI without recompilation.
4. **Semantic clarity:** A multiplier > 1 increases cost (penalty); < 1 decreases cost (incentive).

2 Architecture: Two-Tier Design

The heuristics system splits computation into three evaluation tiers to optimize latency:

2.1 Tier 1: Static Multipliers

Static multipliers depend *only* on request parameters (weather, time, vehicle type, etc.) and are computed **once** when the heuristic closure is built, before any pathfinding begins. Their product is baked into a single float `static_mult`.

$$m_{\text{static}} = m_{\text{weather}} \times m_{\text{time}} \times m_{\text{vehicle}} \times \cdots$$

Static multipliers include:

Heuristic	Input
`weather`	Current weather condition (clear, rain, snow, etc.)
`time_of_day`	Hour and day of week (rush-hour penalties)
`vehicle_type`	Vehicle class (car, motorcycle, bus, etc.)
`visibility`	Visibility in metres (fog penalty)
`humidity`	Relative humidity % (ice risk)
`nightnetwork`	Vehicle type + hour (night-time road availability)
`wind`	Wind speed (m/s) and direction (vehicle-dependent)
`headway`	Vehicle + hour (service frequency for transit)

Example: For a car route at 8 AM on a rainy day:

```
static_mult = 1.0
static_mult *= get_weather_multiplier("rain")          # 1.50
static_mult *= get_time_multiplier(hour=8, dow=1)      # 1.25 (
    rush hour)
static_mult *= get_vehicle_multiplier("car")           # 1.00
# Result: static_mult = 1.875
```

2.2 Tier 2: Goal-Dependent Multipliers

Goal-dependent multipliers depend on the destination node (specifically its district) and are constant throughout a single route search but must be re-evaluated for each new route. They are computed inside the heuristic closure once per call, keyed by the goal district.

Heuristic	Input
`parking`	Goal district (car parking difficulty varies by district)
`holiday_historical`	Date + goal district (holiday traffic patterns)
`events`	Date, hour, goal district (concert/event congestion)
`fiaker`	Goal district, vehicle, hour (horse-carriage zones)
`season`	Month + goal district (seasonal congestion)
`pedestrian_density`	Goal district, vehicle, hour, date (foot traffic)
`delivery`	Goal district, vehicle, hour (delivery-zone restrictions)

Example: A car route to district 1 (innerstadt) at 2 PM on a Friday:

```
mult *= get_parking_multiplier(goal_district="1", vehicle="car")
    # 1.60
mult *= get_pedestrian_multiplier(goal_district="1",
                                  vehicle="car", hour=14, date="
                                     2026-04-21")
```

2.3 Tier 3: Node and Edge-Dependent Multipliers

Edge-dependent multipliers depend on the physical properties of roads at the frontier node (surface, elevation, lighting, etc.) and are the most expensive to compute. To keep the heuristic admissible and avoid iterating all outgoing edges, the system *picks the cheapest outgoing edge as a proxy*:

$$m_{\text{edge}} = \min_{\text{edge} \in \text{adjacency}(n)} m_{\text{surface}}(\text{edge}) \times m_{\text{elevation}}(\text{edge}) \times \cdots$$

This ensures consistency (the heuristic never overestimates from the same node twice) while still reflecting local road quality.

Heuristic	Evaluated On
`surface`	Edge surface material (asphalt, cobblestone, gravel, etc.)
`elevation`	Edge slope (flat vs. uphill penalty)
`safety`	Edge lighting + road class (night-time safety for pedestrians)
`emission_zones`	Edge location (low-emission zones)
`tram_tracks`	Edge surface + weather (tram track penalties in rain/snow)
`snow_priority`	Edge slope + weather (snow clearance priority)
`commuter_bridges`	Edge class + time (congestion on key bridges)
`lane_capacity`	Edge lane count + vehicle (narrow lanes for trucks)
`school_zones`	Edge proximity + time (school zone speed limits)
`scenic`	Edge tags + route profile (scenic routes for tourism)
`road_quality`	Edge condition flag (pothole/repair penalties)
`manual_adjustment`	Edge ID (user-applied overrides)

Node-dependent (not edge-based) multipliers include:

- `intersection_density`: Scans the node's adjacency to count outgoing roads (high density implies slow navigation).
- `road_works`: Checks geospatial proximity to known construction zones.
- `markets`: Proximity to Vienna's weekly farmers' markets (vehicle avoidance).
- `heuriger`: Proximity to wine taverns in Grinzing (pedestrian attraction).

3 The Orchestrator: combined.py

3.1 Module Structure

`combined.py` is the single point of entry for the heuristics system. It exports:

```
def make_heuristic(params: dict) -> Callable:
    """Return a closure h(node_id, goal_id, graph, params)."""
```

The closure returned by `make_heuristic()` is passed to all eight pathfinding algorithms and called hundreds or thousands of times per route.

3.2 Enabling/Disabling Heuristics

The UI sends an optional `enabled_heuristics` list. The interpretation rule is:

- **Empty list:** All 30 heuristics active (baseline behaviour).
- **Non-empty list:** *Whitelist* — only heuristics in the list are active.

```
def _enabled(enabled: set[str], key: str) -> bool:
    return not enabled or key in enabled
```

This allows the UI to toggle heuristics on/off without backend recompilation.

3.3 Closure Construction Pseudocode

```
def make_heuristic(params):
    # Extract all parameters
    enabled = set(params.get("enabled_heuristics", []) or [])
    vehicle, hour, date, weather, etc. = params[...]

    # Infer black ice risk from temperature + humidity
    weather = infer_black_ice(temperature, humidity, weather)

    # Build static multiplier (once)
    static_mult = 1.0
    if _enabled(enabled, "weather"):
        static_mult *= get_weather_multiplier(weather)
    if _enabled(enabled, "time_of_day"):
        static_mult *= get_time_multiplier(hour, day_of_week)
    # ... (8 more static heuristics)

    # Define closure
    def heuristic(node_id, goal_id, graph, params=None):
        h_base = haversine(node_lat, node_lon, goal_lat, goal_lon
            )
        if h_base == 0:
            return 0.0

        mult = static_mult

```

```
        # Goal-dependent tier
        if _enabled(enabled, "parking"):
            mult *= get_parking_multiplier(goal_district, vehicle
                )
        # ... (6 more goal-dependent heuristics)

        # Node-dependent tier
        if _enabled(enabled, "intersection_density"):
            mult *= get_intersection_penalty(node_id, graph)
        # ... (3 more node-dependent heuristics)

        # Edge-dependent tier (pick cheapest outgoing edge)
        best_edge_mult = inf
        for neighbor in adjacency[node_id]:
            edge = edges[neighbor["edge_idx"]]
            e_mult = 1.0
            if _enabled(enabled, "surface"):
                e_mult *= get_surface_multiplier(edge, vehicle)
            # ... (11 more edge-dependent heuristics)
            best_edge_mult = min(best_edge_mult, e_mult)

        if best_edge_mult != inf:
            mult *= best_edge_mult

        h = h_base * mult

        # Cap to preserve admissibility
        return min(h, h_base * MAX_HEURISTIC_CAP)

    return heuristic
```

3.4 Admissibility and Capping

A* requires an *admissible* heuristic that never overestimates true cost:

$$h(n) \leq \text{true cost}(n \to \text{goal})$$

However, weighted A* (weight > 1) relaxes this to *bounded suboptimality*:

$$\text{cost(solution)} \leq \text{weight} \times \text{optimal cost}$$

To remain bounded, the system applies a hard cap:

```
MAX_HEURISTIC_CAP = float(os.getenv("MAX_HEURISTIC_CAP", "50.0"))
return min(h, h_base * MAX_HEURISTIC_CAP)
```

This ensures that even if multipliers stack to extreme values (e.g., heavy snow + night + cobblestone + uphill), the heuristic never exceeds 50× the raw Haversine distance. The default is 50; increase to allow more aggressive heuristics (risking larger suboptimality gaps), or lower to stay closer to admissibility.

4 Heuristic Categories and Patterns

4.1 Pattern 1: Lookup Tables (Static)

The simplest pattern: a dictionary of condition to multiplier.

4.1.1 Example: weather.py

```
WEATHER_MULTIPLIERS = {
    "clear":        1.00,
    "rain":         1.50,
    "heavy_rain":   1.55,
    "snow":         1.60,
    "black_ice":    2.00,
}

def get_weather_multiplier(weather: str) -> float:
    return WEATHER_MULTIPLIERS.get(weather, 1.0)
```

Interpretation: Black ice is 2× slower than clear conditions. Rain is 1.5× slower.

4.1.2 Example: vehicle_type.py

```
VEHICLE_MULTIPLIERS = {
    "car":        1.00,
    "motorcycle": 0.85,  # Smaller, nimbler
    "taxi":       1.05,  # More traffic awareness
    "bus":        1.30,  # Larger, must avoid narrow lanes
    "walking":    1.80,  # Slower base speed
    "bicycle":    1.20,  # Moderate speed
}
```

Motorcycles are faster (0.85); buses and walkers are slower (1.30, 1.80).

4.2 Pattern 2: Conditional Penalties (Static + Dynamic)

A heuristic that returns 1.0 (no penalty) in most cases but applies a multiplier in specific conditions.

4.2.1 Example: safety.py

```
def get_safety_multiplier(edge: dict, hour: int, vehicle: str) ->
    float:
    if vehicle != "walking":
        return 1.0
    if not (hour >= 21 or hour < 5):
        return 1.0  # Daytime: no penalty

    lit = edge.get("lit", "unknown")
    rt = edge.get("road_type", "")
    if lit != "yes":
```

```
        return 1.5  # Unlit road at night: penalty
    if rt in ("primary", "secondary", "tertiary"):
        return 0.9  # Lit primary road at night: incentive
    return 1.1
```

Logic: Only apply night-safety penalties to pedestrians. Prefer lit primary roads at night (multiplier 0.9), penalize unlit roads (1.5), and use a neutral factor for other lit roads (1.1).

4.3 Pattern 3: Multi-Dimensional Lookup (Edge + Vehicle)

A heuristic that combines edge properties with vehicle type via nested dictionaries.

4.3.1 Example: surface.py

```
SURFACE_PENALTIES = {
    "cobblestone": {"car": 1.05, "motorcycle": 1.10, "bicycle":
       1.60, ...},
    "gravel":      {"car": 1.15, "motorcycle": 1.20, "bicycle":
       1.80, ...},
    "unpaved":     {"car": 1.15, "motorcycle": 1.20, "bicycle":
       1.80, ...},
}

def get_surface_multiplier(edge: dict, vehicle: str) -> float:
    surface = edge.get("surface", "asphalt")
    if surface in ("asphalt", "paved", "concrete"):
        return 1.0  # Baseline
    return SURFACE_PENALTIES.get(surface, {}).get(vehicle, 1.0)
```

Interpretation: Bicycles are much slower on gravel (1.80) than cars (1.15). Motorcycles are slower than cars on cobblestone (1.10 vs. 1.05).

4.4 Pattern 4: Geospatial Lookup (District-Based)

Heuristics that key off the Vienna district of a node or edge.

4.4.1 Example: parking.py

```
DISTRICT_PARKING_PENALTY = {
    "1": 1.60,  # Innerstadt (central) --- very difficult
    "2": 1.35,
    "3": 1.25,
    ...
}

def get_parking_multiplier(goal_district: str, vehicle: str) ->
   float:
    if vehicle not in ("car", "truck", "taxi"):
        return 1.0
    return DISTRICT_PARKING_PENALTY.get(goal_district, 1.05)
```

Interpretation: Parking a car in district 1 is 1.6× the cost of district 3 because of limited street parking and high garage fees.

4.5 Pattern 5: Time-Based Conditional (Hour, Day, Date)

Heuristics that respond to temporal conditions (rush hour, weekends, holidays).

4.5.1 Example: time_of_day.py

Typically returns multipliers based on:

- Hour (0—23): Rush hours (8 AM, 5 PM) incur multipliers > 1.
- Day of week (1—7): Weekends may have different patterns.

4.5.2 Example: school_zones.py

```
def get_school_multiplier(edge: dict, hour: int, minute: int, dow: int) -> float:
    if not edge.get("school", False):
        return 1.0
    # Penalize school zones during school hours (7-9 AM, 11-1 PM, 3-5 PM)
    if (7 <= hour < 9) or (11 <= hour < 13) or (15 <= hour < 17):
        if dow not in (6, 7):  # Weekdays only
            return 1.2
    return 1.0
```

4.6 Pattern 6: Proximity-Based (Geospatial Queries)

Heuristics that query the graph for nearby points of interest.

4.6.1 Example: road_works.py

```
def get_road_works_multiplier(lat: float, lon: float, date: str) -> float:
    # Check if node is within 200m of known construction zone
    # and if construction is active on this date
```

4.6.2 Example: markets.py

Similar: checks if the node is near a farmers' market that operates on the given day and hour, and returns a penalty (e.g., 1.3) to avoid congestion.

4.7 Pattern 7: Inference (Derived Conditions)

Heuristics that infer a condition from multiple parameters.

4.7.1 Example: weather.py — Black Ice Inference

```python
def infer_black_ice(temp: float, humidity: float, weather: str) -> str:
    """Promote clear/fog to black_ice when cold + humid."""
    if temp < 2 and humidity > 80 and weather in ("clear", "cloudy", "fog"):
        return "black_ice"
    return weather
```

Logic: When temperature drops below 2 degrees C and humidity exceeds 80%, the system automatically upgrades the weather condition from clear or fog to black ice (multiplier 2.00).

5 The 30 Heuristics: Full Reference

All 30 heuristics are registered in `combined.py` in the `ALL_HEURISTIC_IDS` set. Below is a categorized reference.

5.1 Environmental (5 heuristics)

ID	Module	Description
weather	weather.py	Rain, snow, fog, black ice
visibility	visibility.py	Fog/mist distance penalty
humidity	humidity.py	Moisture-related road conditions
wind	wind.py	Wind speed and direction (vehicle-dependent)
season	season.py	Seasonal road state (salt, leaf cover, etc.)

5.2 Temporal (5 heuristics)

ID	Module	Description
time_of_day	time_of_day.py	Rush-hour congestion (8 AM, 5 PM, etc.)
holiday_historical	holiday_historical.py	Holiday traffic patterns by district
events	events.py	Concerts, sports events, street fairs
headway	headway.py	Transit service frequency (buses, trams)
nightnetwork	nightnetwork.py	Reduced road availability at night

5.3 Vehicle-Type (3 heuristics)

ID	Module	Description
vehicle_type	vehicle_type.py	Base speed by vehicle (car vs. bus vs. walking)
vehicle_type_routing	vehicle_type_routing.py	Vehicle-specific forbidden roads
delivery	delivery.py	Delivery-zone restrictions (time/vehicle)

5.4 Road Geometry and Condition (6 heuristics)

ID	Module	Description
surface	surface.py	Asphalt vs. cobblestone vs. gravel
elevation	elevation.py	Uphill penalty (by slope percentage)
lane_capacity	lane_capacity.py	Multi-lane roads preferred for trucks
tram_tracks	tram_tracks.py	Tram in the road surface (rain/snow penalty)
snow_priority	snow_priority.py	Priority snow-clearing (main roads)
road_quality	road_quality.py	Potholes, repairs, degraded surfaces

5.5 Safety and Regulation (6 heuristics)

ID	Module	Description
safety	safety.py	Lighting and road type at night (pedestrians)
school_zones	school_zones.py	School-hour speed limits
emission_zones	emission_zones.py	Low-emission zone entry penalties
commuter_bridges	commuter_bridges.py	Bridge congestion (rush-hour peaks)
road_works	road_works.py	Construction zones (geospatial + temporal)
manual_adjustment	manual_adjustment.py	User-applied edge overrides

5.6 Urban Features and Attractions (5 heuristics)

ID	Module	Description
intersection_density	intersection_density.py	High density = slow navigation
pedestrian_density	pedestrian_density.py	Foot traffic at destination (hour/date)
markets	markets.py	Farmers' markets (weekly schedule)
heuriger	heuriger.py	Wine taverns in Grinzing (seasonal/hourly)
fiaker	fiaker.py	Horse-carriage zones (tourist routes)

5.7 Destination-Specific (3 heuristics)

ID	Module	Description
parking	parking.py	Parking difficulty by district (cars only)
scenic	scenic.py	Prefer scenic routes (if route_profile = scenic)
(reserved)		(no additional destination-specific slots)

Total: 30 heuristics.

6 Adding a New Heuristic

To add a new heuristic (e.g., noise pollution avoidance):

1. **Create module:** Write `src/heuristics/noise.py` with:

```python
def get_noise_multiplier(edge: dict, hour: int) -> float:
    if hour >= 22 or hour < 6:
        return 1.0  # Night: traffic already reduced

    road_type = edge.get("road_type", "")
    if road_type in ("motorway", "trunk"):
        return 1.3  # High noise on major roads
    return 1.0
```

2. **Import in combined.py:**

```python
from src.heuristics.noise import get_noise_multiplier
```

3. **Register ID:** Add to `ALL_HEURISTIC_IDS`:

```python
ALL_HEURISTIC_IDS = {
    ..., "noise", ...
}
```

4. **Place in correct tier:** For edge-dependent heuristics, add in the edge loop:

```python
if _enabled(enabled, "noise"):
    e_mult *= get_noise_multiplier(edge, hour)
```

   For static or goal-dependent heuristics, place in the appropriate section.

5. **Test:** The heuristic is immediately available via the UI's `enabled_heuristics` list.

7 Performance Considerations

7.1 Closure-Freeing Optimization

The closure-building pattern (static tier computed once, dynamic tier reused) is critical for performance:

```
# Expensive: re-compute weather every call
def bad_heuristic(node, goal, graph, params):
    weather = params.get("weather")
    mult = get_weather_multiplier(weather)  # Called 1000s of
        times!
    ...

# Efficient: pre-compute static tier
def make_good_heuristic(params):
    weather_mult = get_weather_multiplier(params.get("weather"))
    def heuristic(node, goal, graph, params=None):
        mult = weather_mult  # Already computed
        ...
    return heuristic
```

The Vienna graph has approximately 1000 nodes; A* explores approximately 300–500 nodes per route. With 8 algorithms running in parallel, the heuristic is called approximately 2400–4000 times per pathfinding request. Precomputing the static tier (8 multiplications) saves approximately 16000–32000 multiplications per request.

7.2 Edge Proxy Strategy

Iterating all outgoing edges for each frontier node is expensive:

$$\text{Cost per call} \approx O(\text{degree}(n) \times \text{num enabled heuristics})$$

With an average degree of approximately 3 and 15 edge-dependent heuristics, each frontier node incurs approximately 45 heuristic evaluations. The proxy strategy (pick the cheapest edge) reduces this to a single pass.

7.3 Capping Mechanism

The hard cap ensures bounded suboptimality without re-evaluating the entire multiplier stack:

```
return min(h, h_base * MAX_HEURISTIC_CAP)
```

This is $O(1)$ and prevents pathological blow-up when many heuristics stack.

8 Integration with Pathfinding Algorithms

All eight algorithms have the same interface:

```
def find_path(graph, start_id, goal_id, heuristic_fn, params) ->
    PathResult
```

The heuristic function is called as:

```
h_value = heuristic_fn(node_id, goal_id, graph, params)
```

8.1 Admissibility Requirements

- **BFS, DFS, UCS:** Ignore the heuristic; no admissibility constraint.
- **Dijkstra:** Ignores the heuristic (uses only $g(n)$).
- **Greedy Best-First:** Uses heuristic as the only criterion; admissibility not required.
- **A*:** Requires an admissible heuristic $h(n) \leq \text{true cost}(n \to \text{goal})$.
- **Weighted A*:** Allows bounded-suboptimal heuristics (multiplied by a weight factor).
- **Bidirectional A*:** Requires admissibility in both directions.

The static tier multipliers (weather, vehicle, time-of-day) are designed to be admissible for A*. The edge-dependent tier (surface, elevation) reflects actual road properties and remains admissible. The hard cap (`MAX_HEURISTIC_CAP`) ensures weighted A* stays bounded-suboptimal.

9 Configuration and Testing

9.1 Environment Variables

- `MAX_HEURISTIC_CAP`: Maximum multiplier cap (default: 50.0). Lower for admissibility; raise for more aggressive heuristics.
- No other heuristic-specific environment variables.

9.2 Testing a Heuristic

To test a new heuristic without recompiling:

```
# In a test script
from src.graph.loader import load_graph
from src.heuristics.combined import make_heuristic

graph = load_graph()
params = {
    "weather": "snow",
    "vehicle_type": "car",
```

```
    "hour": 8,
    ...
}

h_fn = make_heuristic(params)
test_node = "some_node_id"
goal_node = "goal_id"

h_value = h_fn(test_node, goal_node, graph, params)
print(f"Heuristic value: {h_value}")
```

10 Summary

The heuristics system in `src/heuristics/` is a modular, efficient, and interactive cost model that bridges the gap between pure distance-based pathfinding and real-world routing.

10.1 Key Design Points

1. **Three-tier evaluation:** Static (pre-computed), goal-dependent (per-route), and edge-dependent (per-frontier-node).
2. **Multiplicative composition:** Individual multipliers combine via multiplication, not addition.
3. **Closure factory pattern:** Pre-computing static multipliers into a closure achieves sub-millisecond heuristic latency.
4. **Edge proxy strategy:** Checking only the cheapest outgoing edge avoids iterating all neighbors.
5. **Admissibility capping:** A hard cap ensures A* and weighted A* remain bounded-suboptimal.
6. **Modularity:** 30 independent heuristics can be toggled via the UI without recompilation.

10.2 File Organization

- `combined.py`: Orchestrator; closure factory; two-tier builder; edge-loop iterator.
- `*.py` (30 modules): Individual heuristics; each exports a single `get_*_multiplier()` function.
- `__init__.py`: Module metadata (currently minimal).

10.3 Code Style

All heuristics follow consistent patterns:

- Input: edge dict or parameters; output: multiplier float.
- No side effects; pure functions.
- Default multiplier: 1.0 (no penalty).
- Multiplier > 1: penalty (slower); < 1: incentive (faster).